SYNAPSE BUILDOS: GENERATIVE AI-POWERED RESIDENTIAL ARCHITECTURAL DESIGN, ESTIMATION, AND COMPLIANCE PLATFORM

A MINI PROJECT REPORT

Submitted by

PROJECT TEAM
(REGISTER NUMBER: 9124104073 / 9124104103 / 9124104121)

in partial fulfilment for the award of the degree
of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING

KCG COLLEGE OF TECHNOLOGY, KARAPAKKAM
(AUTONOMOUS)
NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this mini project report **"SYNAPSE BUILDOS: Generative AI-Powered Residential Architectural Design, Estimation, and Compliance Platform"** is the bonafide work of **"Project Team"** who carried out the mini project work under my supervision.

Signature of the HOD with date

Dr. S. REVATHY M.E., Ph.D.,
PROFESSOR AND HEAD,
Department of Computer Science and Engineering,
KCG College of Technology,
Karapakkam, Chennai.

Signature of the Supervisor with date

Dr. ROBINSON JOEL
ASSOCIATE PROFESSOR,
Department of Computer Science and Engineering,
KCG College of Technology,
Karapakkam, Chennai.

Certified that the above candidate(s) was examined in the End Semester mini-Project Viva-Voce Examination held on .....

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION BY THE STUDENT

We hereby declare that this mini project report titled **“SYNAPSE BUILDOS: Generative AI-Powered Residential Architectural Design, Estimation, and Compliance Platform”**, under the guidance of **“Dr. Robinson Joel”** is the original work done by us and we have not plagiarized or submitted to any other degree in any university by us.

STUDENT CANDIDATES
Department of Computer Science and Engineering
KCG College of Technology, Chennai

ACKNOWLEDGEMENT

We thank the Almighty GOD for the abundant blessings showered on us. We extend our deepest love and gratitude to our dear parents who built up our career and backed us up in life.

We thank our management and our Principal **Dr. G. Prabhakaran** for the opportunities given to us for our career development.

We feel indebted to the Head of the Department **Dr. Revathy S**, Professor and Head, Department of Computer Science & Engineering, KCG College of Technology, Karapakkam, for all her encouragement, which has sustained our labor and efforts.

We express our deepest gratitude to the internal guide **Dr. Robinson Joel**, Associate Professor, Department of Computer Science & Engineering, KCG College of Technology, for his valuable guidance, ideas and support.

We would like to thank all other faculty members of Department of Computer Science & Engineering, for their help and advice throughout our life in this campus.

Finally, we are thankful to all our friends and all others who encouraged us and helped us in doing this project.

PROJECT CANDIDATES

ABSTRACT

Synapse BuildOS is an intelligent residential architectural design, cost estimation, and statutory compliance platform developed using modern web standards (HTML5, SVG, Three.js, TypeScript) and high-performance Python backend architectures (FastAPI, Pydantic, SQLAlchemy, Anthropic Claude LLM). The platform is engineered to eliminate the severe real-world disconnect between architectural drafting, civil engineering quantity estimation, and municipal bylaw approval.

In Synapse BuildOS, room geometry serves as the immutable single source of truth. Users can ingest project briefs from scanned documents, PDFs, or natural language prompts across English, Malayalam, Hindi, and Tamil. The system recommends pre-designed regional floor plans and provides an interactive 2D SVG plan editor where users can manipulate rooms in real coordinates (feet) on true plot boundaries with automated setback buffers. Every modification instantly updates the corresponding 3D volumetric massing model with interactive walkthroughs and sun-path simulations.

Crucially, Synapse BuildOS features an automated rule-verification engine that continuously evaluates designs against the **Tamil Nadu Combined Development and Building Rules (TNCDBR 2019)**, checking Floor Space Index (FSI), ground coverage, setbacks, parking provisions, room dimensions, and vertical access. Simultaneously, an arithmetic cost engine computes line-item civil estimates directly against the published **Tamil Nadu PWD Plinth Area Rates (2025–26)**, guaranteeing that cost is strictly deterministic arithmetic rather than hallucinated projections.

The platform aggregates these dimensions into a comprehensive **Construction Readiness Score (0–100)** spanning budget fit, regulatory compliance, buildability, and sustainability. Users can generate publication-grade PDF project dossiers and export native CAD/BIM formats (IFC4, DXF, OBJ, CSV BOQ). By unifying geometric modeling, civil engineering estimation, and generative AI copilots into a zero-build browser environment, Synapse BuildOS drastically shortens the residential pre-construction lifecycle from weeks to seconds.

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Comparative Analysis of Existing Architectural & Construction Systems	15
3.4.1	Tools and Technologies Used in Synapse BuildOS	24
4.1	Geometric Takeoff and Rule Verification Validation Results	30

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	Synapse BuildOS Platform Overview	11
3.1	Synapse BuildOS System Architecture	20
3.2	System Flow Diagram	22
3.3	Data Flow Diagram (Level 0 Context & Level 1 Detailed DFD)	23
3.5	UML Use Case Diagram	25
4.4	Synapse BuildOS 3D Spatial Models (Exterior & Axonometric)	31
4.5	Sample Architectural Design Model (Villa Harmony 3BHK Duplex — 2D Plan & 3D Massing)	32

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
CAD	Computer-Aided Design
BIM	Building Information Modeling
AEC	Architecture, Engineering, and Construction
TNCDBR	Tamil Nadu Combined Development and Building Rules
PWD	Public Works Department (Tamil Nadu)
FSI	Floor Space Index
BOQ	Bill of Quantities
AABB	Axis-Aligned Bounding Box
SVG	Scalable Vector Graphics
API	Application Programming Interface
UI	User Interface
UX	User Experience
OCR	Optical Character Recognition
LLM	Large Language Model
REST	Representational State Transfer
IFC	Industry Foundation Classes (IFC4)
DXF	Drawing Exchange Format
ORM	Object-Relational Mapping (SQLAlchemy)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	5
	LIST OF TABLES	6
	LIST OF FIGURES	7
	LIST OF ABBREVIATIONS	8
1.	**INTRODUCTION**	**11**
1.1	Overview	11
1.2	Motivation	12
1.3	Objective	12
1.4	Scope	13
1.5	Significance of the study	13
2.	**LITERATURE SURVEY**	**14**
2.1	Existing systems and comparative analysis	14
2.2	Computational geometry and generative AI in residential CAD	16
2.3	Automated building bylaw compliance and PWD estimation	17
2.4	Research gap	17
3.	**SYSTEM DESIGN AND IMPLEMENTATION**	**19**
3.1	System overview	19
3.2	System architecture	19
3.3	System flow	21

TABLE OF CONTENTS (CONTINUED)

CHAPTER NO.	TITLE	PAGE NO.
3.4	Data flow diagram (DFD)	22
3.4.1	Tools and technologies used	24
3.5	Use case diagram	25
3.6	System modules description	25
3.7	Implementation detail	26
3.8	Implementation process	27
3.9	Advantages of the system	27
3.10	Challenges faced during implementation	28
4.	**TESTING AND RESULT**	**29**
4.1	Introduction	29
4.2	Types of testing	29
4.3	Testing procedures	30
4.4	Results	31
4.5	Conclusion of testing	32
5.	**CONCLUSION AND FUTURE WORK**	**33**
5.1	Conclusion	33
5.2	Future work	34
	APPENDIX 1: SAMPLE CODE	**35**
	REFERENCES	**48**

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Synapse BuildOS is an intelligent, full-stack architectural design, estimation, and statutory compliance platform developed using modern web technologies (HTML5, Scalable Vector Graphics, JavaScript/TypeScript, Three.js) and Python backend computational frameworks (FastAPI, Pydantic, SQLAlchemy, Anthropic Claude). It is explicitly designed to eliminate the historical divide between architectural drafting, civil engineering cost estimation, and municipal regulatory approval.

In conventional residential construction, drafting a floor plan in AutoCAD, calculating quantity takeoff in Microsoft Excel, and verifying compliance against local municipal building codes (such as the Tamil Nadu Combined Development and Building Rules, TNCDBR 2019) are conducted by separate professionals in isolated silos. When a client requests a modification—such as expanding a master bedroom or moving a living room partition—the entire cycle of drafting, structural resizing, cost re-estimation, and bylaw checking must be manually repeated. This latency causes severe budget overruns, project delays, and frequent statutory rejections.

Synapse BuildOS resolves this systemic friction by establishing **room geometry as the single source of truth**. A room is modeled as an atomic rectangle with spatial coordinates defined in feet. From these exact rectangles, built-up areas, wall surface areas, door and window schedules, Floor Space Index (FSI), plot coverage, setback buffers, PWD plinth-area construction costs, and 3D volumetric models are computed instantaneously and deterministically.



Figure 1.1: Synapse BuildOS Platform Overview

1.2 MOTIVATION

Many homeowners and prospective builders in India undertake residential construction without direct, real-time access to professional architectural planning and compliance advice. Commercial CAD software like AutoCAD and Revit require steep learning curves and expensive licensing, while consumer floor-plan tools remain purely cosmetic, lacking any understanding of local statutory building codes or regional construction rate schedules.

The primary motivation behind Synapse BuildOS is to provide architects, contractors, and homeowners with an accessible, unified residential building intelligence platform. By combining computational 2D/3D geometry, deterministic civil engineering quantity estimation, and generative AI copilot assistance, the system allows non-technical users to explore viable house layouts with immediate, trusted feedback on budget feasibility and legal compliance.

Key motivation factors include:

- Eliminating the disconnect between architectural design and cost estimation.
- Preventing municipal building plan rejections by enforcing TNCDBR 2019 rules in real time.
- Providing deterministic, auditable cost estimation based on published Tamil Nadu PWD rates.
- Supporting natural language design modifications through multi-lingual AI prompts.
- Offering a zero-install, browser-based CAD and 3D walkthrough experience.

1.3 OBJECTIVE

The primary technical objectives of the project are as follows:

1. To establish a unified geometric data model where 2D SVG rectangles serve as the single source of truth for all spatial, structural, and cost metrics.
2. To develop an automated statutory compliance engine that validates residential designs against the Tamil Nadu Combined Development and Building Rules (TNCDBR 2019).
3. To implement a deterministic quantity takeoff and cost estimation engine based on the published Tamil Nadu PWD Plinth Area Rates (2025–26).
4. To generate real-time 3D parametric massing models and first-person walkthroughs using Three.js directly in the browser.
5. To integrate a multi-lingual Large Language Model assistant (Cortex AI) capable of translating conversational client requests into valid geometric adjustments.
6. To compute a comprehensive Construction Readiness Score (0–100) and export professional CAD/BIM assets (IFC4, DXF, OBJ, CSV BOQ, and PDF reports).

1.4 SCOPE

The scope of the Synapse BuildOS project encompasses:

- Design and development of a full-stack, browser-based residential CAD studio.
- Support for residential building typologies (villas, bungalows, compact urban houses, row housing) up to Ground + 2 floors.
- Ingestion of client briefs via scanned documents, images, PDFs (using OCR), and freeform text.
- Real-time 2D floor plan editing with live collision detection and setback boundaries.
- Interactive 3D spatial rendering, material swatches, door/window punch-outs, and sun-path lighting.
- Automated TNCDBR 2019 checks covering FSI, ground coverage, front/rear/side setbacks, room aspect ratios, and parking.
- Real-time civil budget calculation incorporating structural RCC frame, masonry, finishes, plumbing, and electrical points.

1.5 SIGNIFICANCE OF THE STUDY

- **Democratization of Architectural Intelligence:** Enables homeowners to participate actively in the design of their living spaces while understanding cost and legal ramifications.
- **Zero Re-work and Cost Predictability:** Changes to room boundaries reflect immediately across quantity takeoff and PWD pricing, eliminating post-tender budget escalation.
- **Statutory Compliance Assurance:** Proactive violation flags prevent costly penalties, plan submission rejections, and structural demolition risks.
- **AEC Stakeholder Consensus:** Provides role-based views where owners, architects, and contractors share the identical data model without version confusion.
- **Advancement in Applied AI:** Demonstrates how generative LLMs can be constrained by deterministic geometric rules to provide reliable, hallucination-free CAD modifications.
- **Interoperability:** Bridges consumer web applications with professional AEC workflows through standard IFC4 and DXF exports.

CHAPTER 2

LITERATURE SURVEY

2.1 EXISTING SYSTEMS AND COMPARATIVE ANALYSIS

Computer-Aided Design (CAD) and Building Information Modeling (BIM) tools have traditionally focused on high-precision geometric drafting for trained professionals. Commercial platforms such as Autodesk AutoCAD and Revit provide comprehensive drafting suites but require extensive training, specialized workstations, and costly subscriptions. Crucially, these desktop tools lack native awareness of regional Indian municipal bylaws and do not incorporate localized government rate schedules.

Consumer-oriented web tools such as Planner 5D, RoomSketcher, and Sweet Home 3D allow non-professionals to create 2D and 3D floor plans. However, these tools operate purely at a visual and decorative level. They treat walls as abstract graphics rather than structural assemblies, cannot calculate structural civil quantities, and have no capability to verify municipal setbacks or zoning regulations.

Recent advancements in generative AI for architecture (e.g., Finch 3D, Hypar, and generative floor plan research models) employ machine learning to generate spatial layouts. However, most generative research models generate raster images or unconstrained vector geometry that frequently produce overlapping rooms, unbuildable structural configurations, and hallucinated dimensions. They lack deterministic compliance verification against statutory codes such as the Tamil Nadu Combined Development and Building Rules (TNCDBR 2019).

Synapse BuildOS addresses these limitations by introducing a hybrid architecture: a deterministic geometric kernel that computes exact arithmetic for cost and bylaws, paired with a constrained LLM copilot that operates exclusively through validated schema transformations.

Table 2.1: Comparative Analysis of Existing Architectural & Estimation Systems

Existing System	Target Audience	Type	Key Features	Limitations	Remarks
AutoCAD / Revit	Architects, structural engineers	Desktop CAD / BIM Suite	High-precision drafting, 3D modeling, construction docs	Steep learning curve, high cost, no Indian municipal bylaw rules	Industry standard drafting, not accessible for rapid decisions
Planner 5D / RoomSketcher	Homeowners, interior decorators	Consumer Web / Mobile App	Drag-and-drop 2D/3D layouts, furniture catalogue	Purely visual; no civil costing, no bylaw compliance checking	Good for interior visualization, inadequate for building approvals
Finch 3D / Hypar	Enterprise developers, computational architects	Generative BIM Platform	Algorithmic massing, daylight optimization	Geared for commercial high-rises; lacks Indian regional PWD costing	Advanced generative tool, but disconnected from local Indian regulations
Synapse BuildOS (Proposed)	**Homeowners, architects, local builders**	**Web-based Building Intelligence OS**	**Single source of truth, 2D SVG, 3D Three.js, live TNCDBR & PWD cost**	**Focused on residential typologies up to G+2**	**Zero-install, deterministic, auditable, and accessible to all AEC users**

2.2 COMPUTATIONAL GEOMETRY AND GENERATIVE AI IN RESIDENTIAL CAD

Computational geometry serves as the foundational backbone for Computer-Aided Architectural Design. In architectural space planning, floor plans can be conceptualized as planar graphs where rooms represent vertices and shared partitions represent edges. However, maintaining geometric integrity during dynamic user editing introduces challenging computational problems, particularly in collision detection, boundary clipping, and spatial adjacency.

Synapse BuildOS implements an Axis-Aligned Bounding Box (AABB) sweep-and-prune algorithm operating with a calibrated 0.05 ft tolerance. This allows the system to instantaneously detect invalid room overlaps while permitting adjacent rooms to share common partition walls without computational ambiguity. When rooms share an edge, the takeoff engine assigns a 0.5 weighting to the partition run, ensuring that shared walls are not double-counted in civil brickwork estimates.

Recent advances in Large Language Models (LLMs) have enabled conversational software interactions. However, unconstrained LLMs cannot perform reliable spatial reasoning and frequently output geometrically invalid or self-intersecting polygons. In Synapse BuildOS, the generative AI subsystem (Cortex) is decoupled from direct geometric creation. Instead of drawing lines, Claude generates high-level structured mutations (e.g., 'expand_room', 'split_space', 'swap_finishes') validated against strict Pydantic schemas.

These mutations are subsequently passed through the deterministic geometry kernel, which executes the required coordinate transformations, verifies boundary constraints, and recalculates engineering quantities before presenting the updated design to the user.

2.3 AUTOMATED BUILDING BYLAW COMPLIANCE AND PWD ESTIMATION

Statutory compliance verification in Indian civil engineering has traditionally been a manual, post-design inspection process. The **Tamil Nadu Combined Development and Building Rules (TNCDBR 2019)** govern all residential construction within the state, mandating specific geometric constraints based on plot size, road width, and building height. Rules dictate minimum front, rear, and side setbacks, permissible Floor Space Index (FSI), maximum ground coverage, mandatory vehicle parking dimensions, and minimum habitable room areas.

Manual checking of these clauses is prone to human error and frequently delays municipal approvals. Synapse BuildOS codifies the TNCDBR 2019 residential provisions into deterministic mathematical evaluation rules. Every time a room is dragged or resized, the engine re-evaluates setbacks against plot boundaries, recalculates total built-up area and FSI, checks room aspect ratios, and highlights non-compliant conditions with exact numerical margins.

Similarly, construction budgeting is commonly distorted by subjective contractor markups. Synapse BuildOS anchors all financial estimation in the published **Tamil Nadu Public Works Department (TN PWD) Schedule of Rates (2025–26)**. By calculating exact plinth areas, perimeter wall runs, structural RCC column-beam grids, flooring surfaces, and door/window counts, the cost engine applies published government rates line by line, providing an auditable, transparent estimate that can be verified by hand.

2.4 RESEARCH GAP

Despite recent software advancements, significant research gaps persist in architectural CAD:

1. **Fragmented Workflows:** CAD drafting, quantity estimation, and municipal compliance operate as disconnected tools, causing severe information loss between stages.
2. **Lack of Regional Statutory Rules:** Mainstream CAD engines do not encode Indian municipal building rules (TNCDBR, NBC), leaving compliance to manual checking.
3. **Opaque Financial Projections:** Existing tools lack integration with authoritative public rate schedules (PWD), leading to untrusted cost projections.
4. **Unreliable Generative AI:** Generative image models hallucinate unbuildable plans without structural, dimensional, or legal validity.

KEY LIMITATIONS OBSERVED IN EXISTING SYSTEMS

An analysis of current industry practices reveals the following primary bottlenecks:

1. **Technical Complexity:** Advanced CAD and BIM platforms require years of professional training, alienating homeowners and small contractors from direct participation in the design loop.
2. **Delayed Feedback Loops:** Design revisions take days or weeks to be manually re-costed and re-checked for compliance, slowing project initiation.
3. **Absence of Single Source of Truth:** Floor plans, 3D presentations, and BOQ spreadsheets frequently disagree because they are maintained in disparate files.
4. **High Risk of Statutory Rejection:** Violations of municipal setbacks and FSI limits are often discovered only during formal municipal plan scrutiny, necessitating expensive structural redesigns.
5. **High Cost of Software Access:** Enterprise drafting packages impose recurring per-seat licensing fees that are prohibitive for small building practices.
6. **Language Barriers:** Available architectural tools are exclusively English-centric, hindering effective collaboration with regional clients and on-site contractors.

The Synapse BuildOS Solution: Synapse BuildOS addresses each of these gaps by offering a unified, zero-install web platform that couples deterministic geometric modeling with automated TNCDBR bylaw verification, PWD plinth-area rate calculation, multi-lingual AI assistance, and open CAD/BIM data portability.

CHAPTER 3

SYSTEM DESIGN AND IMPLEMENTATION

3.1 SYSTEM OVERVIEW

Synapse BuildOS is structured around a centralized geometric state engine where all spatial, structural, compliance, and financial metrics are derived from a single array of room rectangles. The system features a responsive dual-surface architecture:

1. **Landing Page & Project Brief Configurator:** Ingests user requirements (plot dimensions, budget ceiling, room count, architectural style) through interactive forms, document upload (OCR), or conversational voice prompts.

2. **Visual CAD Studio:** A full-featured design workspace providing interactive 2D SVG plan editing, real-time Three.js 3D extrusion, live TNCDBR 2019 compliance auditing, line-item PWD cost breakdown, and one-click PDF dossier generation.

Because the coordinate space in the SVG canvas uses feet as its native units, the visual representation is mathematically identical to the internal analytical model. No translation layers exist to introduce rounding errors or coordinate drift.

3.2 SYSTEM ARCHITECTURE

Synapse BuildOS implements a multi-tiered architecture comprising five coordinated subsystems:

- **User Interface Layer:** Built with HTML5, CSS3, SVG, and Vanilla JavaScript / React, requiring zero bundler toolchain dependencies. It executes high-speed pointer-driven canvas operations at 60 FPS.
- **Client-Side Engine Kernel (web/engine.js):** Contains the shared mathematical model, geometry takeoff routines, TNCDBR compliance predicates, and PWD cost formulas.
- **FastAPI Backend Services:** High-throughput Python microservices providing authentication, persistent project storage, PDF dossier compilation, and CAD/BIM exporters (IFC4, DXF).
- **Generative Intelligence (Cortex AI):** Powered by Anthropic Claude 3.5, providing conversational design guidance, multi-lingual translation, and rule-constrained plan mutation.
- **Data Persistence:** SQLite database for development and PostgreSQL for production, managed through SQLAlchemy 2.0 ORM with Redis caching.

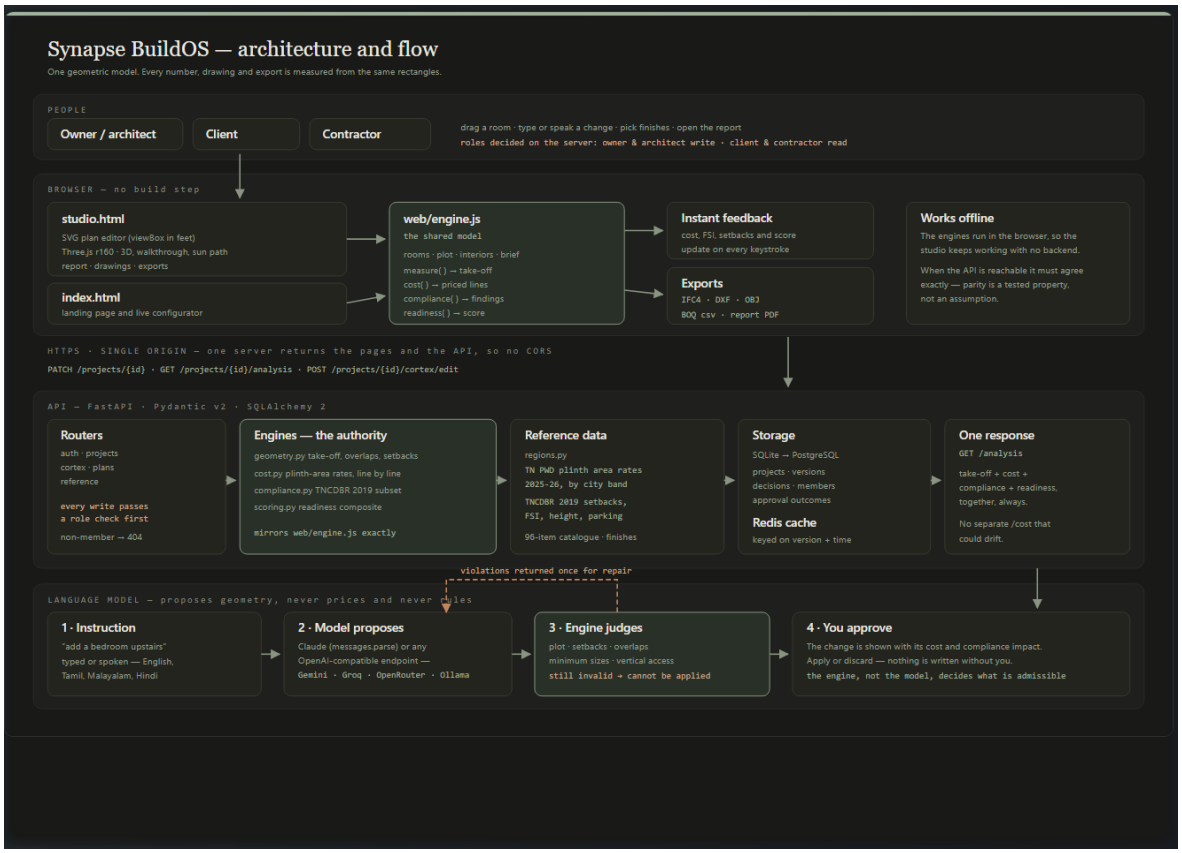


Figure 3.1: Synapse BuildOS System Architecture

Figure 3.1 illustrates the comprehensive system architecture of Synapse BuildOS. The architecture enforces strict separation of concerns across AEC stakeholders (Owner, Architect, Contractor), the browser execution environment, the shared mathematical model, and the server-side computational engines. By maintaining the room rectangles as the authoritative atom of state, all client-side viewports and server-side analysis microservices remain perfectly synchronized.

3.3 SYSTEM FLOW

The end-to-end operational workflow of Synapse BuildOS proceeds across nine distinct stages:

1. **Start Application:** The user initializes the Synapse BuildOS studio in any modern browser without requiring plugin installations or local drafting software.

2. **Input Client Brief:** The user defines plot dimensions (width and depth in feet), budget ceiling, target location (e.g., Chennai, Coimbatore, Kochi), and required room typologies via text or document upload.

3. **Select Recommended Plan:** The recommendation engine queries the library of pre-engineered typologies and selects the layout with the highest match score against plot and family criteria.

4. **Render 2D Plan Geometry:** The system renders the floor plan on an SVG canvas in feet, automatically drawing municipal setback boundaries, room rectangles, and partition walls.

5. **Extrude 3D Spatial Walkthrough:** The Three.js pipeline extrudes 2D room boundaries into 3D massing blocks, procedural wall textures, fenestrations (doors/windows), and realistic ambient lighting.

6. **Compute Geometric Takeoff:** The geometry kernel calculates exact built-up area, ground footprint, perimeter wall surface runs, flooring area, and opening deductions.

7. **Evaluate Compliance & PWD Cost:** The system checks TNCDBR 2019 rules (setbacks, coverage, FSI, height) and calculates construction cost line by line against the published TN PWD schedule.

8. **Calculate Construction Readiness Score:** The scoring module synthesizes budget fit, compliance, buildability, and sustainability into a 0–100 readiness index.

9. **Generate Dossier & Export CAD/BIM:** The user exports production assets including PDF architectural reports, IFC4 BIM files, 2D DXF CAD drawings, OBJ 3D meshes, and CSV BOQ schedules.

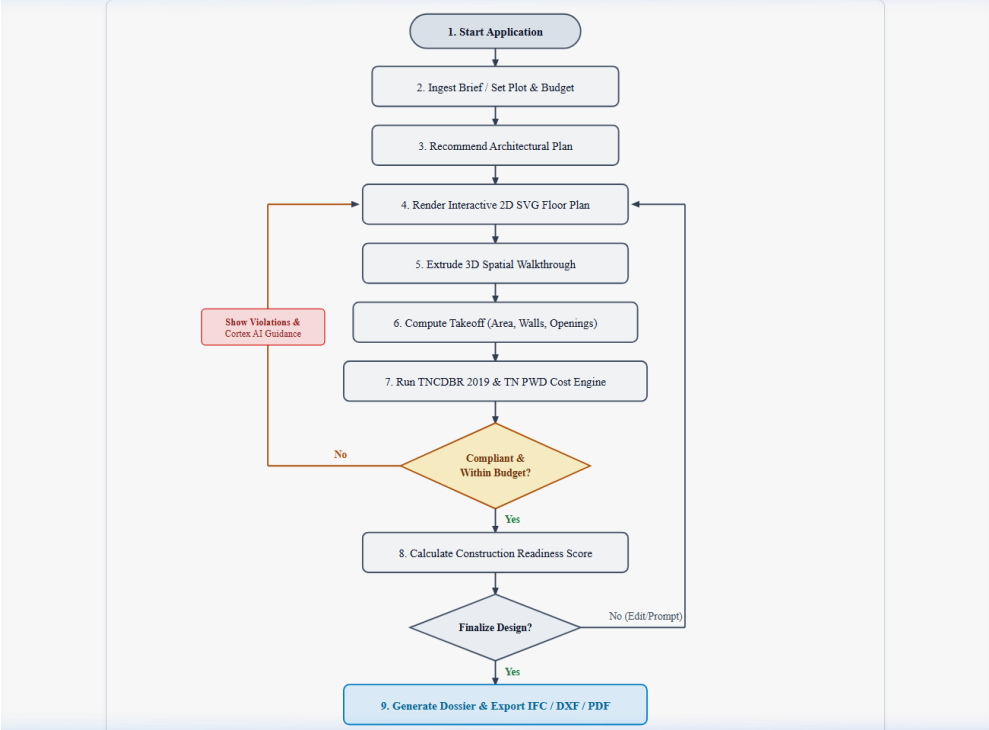


Figure 3.2: System Flow Diagram

3.4 DATA FLOW DIAGRAM (DFD)

The Data Flow Diagram illustrates how spatial data, user modifications, and statutory rate cards traverse the Synapse BuildOS system across conceptual and detailed levels.

Level 0: Context Diagram: The user (Homeowner, Architect, or Contractor) supplies brief parameters, spatial adjustments, and AI prompts to the Synapse BuildOS core engine. The system processes these inputs and returns real-time 2D/3D visualizations, TNCDBR compliance audit checklists, PWD cost estimates, and CAD/BIM export packages.

Level 1: Detailed DFD: The system deconstructs data flow into discrete functional transformations: Brief Ingestion (1.0), 2D SVG Modeling (2.0), 3D Massing Extrusion (3.0), Compliance Verification (4.0), Cost Takeoff (5.0), and Readiness Scoring (6.0), coordinated with live databases and rate cards.

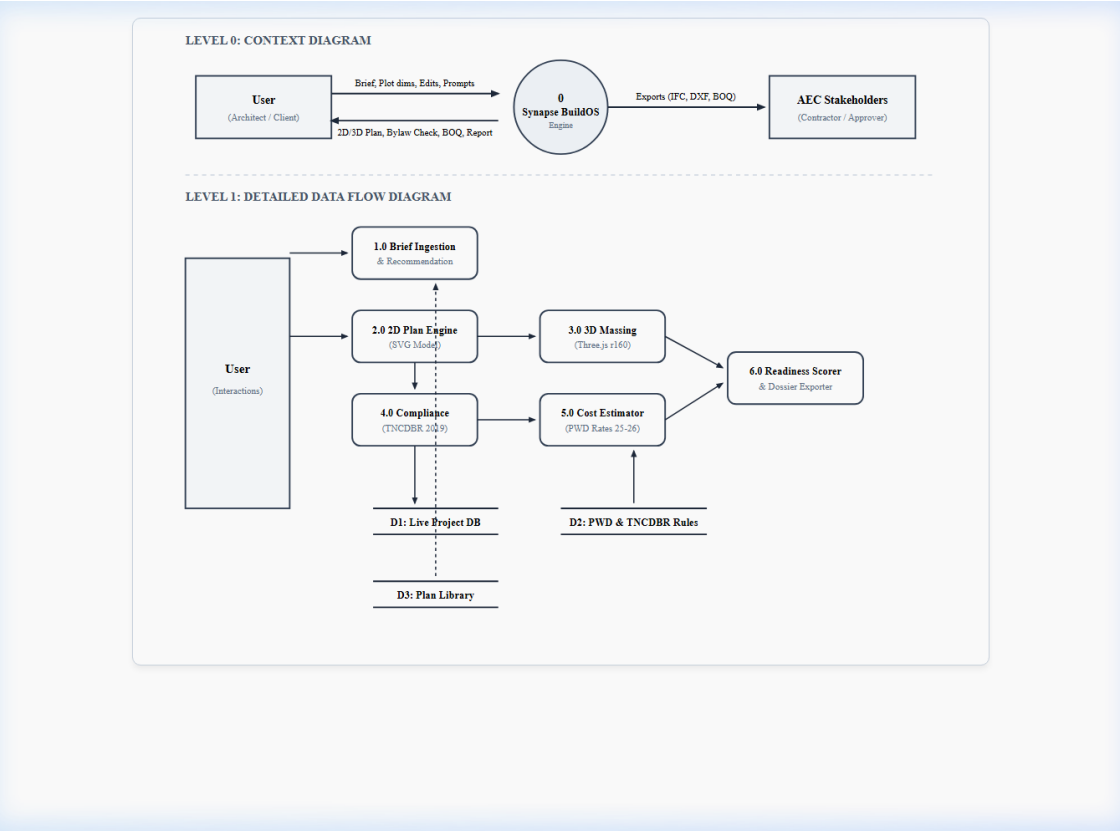


Figure 3.3: Data Flow Diagram (Level 0 & Level 1)

As shown in Figure 3.3, Level 0 models the external interaction boundaries between users and the core engine. Level 1 maps internal information flows between processes and primary data stores: **D1 (Project DB)**, **D2 (PWD Rate Cards & TNCDBR Rules)**, and **D3 (Plan Library)**. Because each process queries standardized data models, calculations remain fully deterministic.

3.4.1 TOOLS AND TECHNOLOGIES USED

Component	Technology Used	Description
Front-end Architecture	HTML5, CSS3, Vanilla JS / TypeScript, SVG	Builds the responsive, zero-build web interface and 2D canvas editor
3D Graphics & Walkthrough	Three.js r160, WebGL	Renders procedural 3D massing, materials, first-person camera, and sun shadows
Document Processing / OCR	Tesseract.js, PDF.js	Extracts client briefs, plot dimensions, and space requirements from scanned files
Geometric Engine	JavaScript (web/engine.js) & Python Shapely	Executes AABB collision detection, area takeoff, and perimeter wall calculations
Compliance Engine	JavaScript / Python	Evaluates rules against Tamil Nadu Combined Development & Building Rules (TNCDBR 2019)
Cost Estimation Engine	Python / JavaScript	Computes line-item civil estimates based on published TN PWD 2025–26 rates
Backend Framework	FastAPI, Python 3.11+	High-speed REST API for project management, exports, and LLM orchestration
Database & ORM	SQLite / PostgreSQL, SQLAlchemy 2.0	Persists projects, immutable snapshots, audit logs, and regional rate cards
AI Copilot Subsystem	Anthropic Claude 3.5 Sonnet (Cortex Engine)	Translates multi-lingual voice/text prompts into validated schema mutations
CAD/BIM Export Engine	IfcOpenShell, DXF Generator	Exports industry-standard IFC4 building models and AutoCAD DXF drawings
Development IDE & VCS	Visual Studio Code, Git, GitHub	Used for version control, continuous integration, and collaborative development

3.5 USE CASE DIAGRAM

The Use Case Diagram depicts the functional interactions between system actors (Homeowner/Client, Architect/Designer, Contractor/Estimator) and the Synapse BuildOS platform.

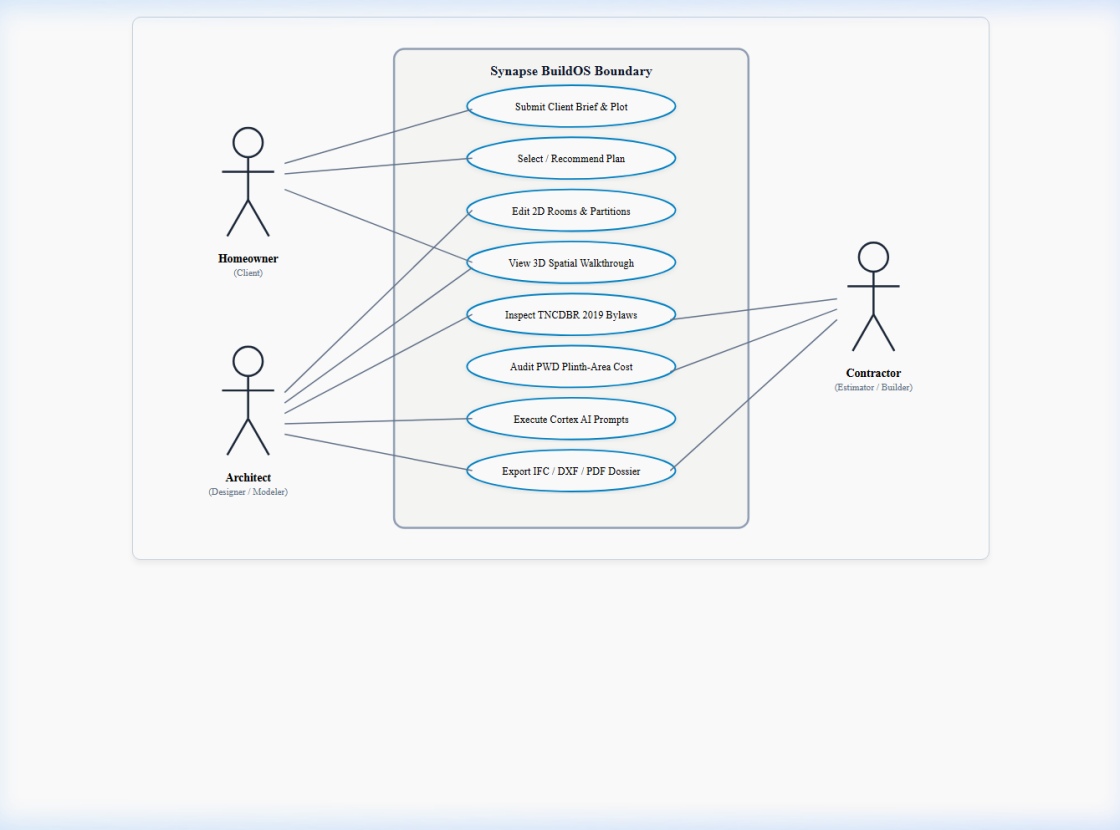


Figure 3.5: Use case diagram

Actors and Roles:

- **Homeowner (Client):** Submits briefs, explores recommended layouts, inspects budget fit, and takes 3D walkthroughs.
- **Architect (Designer):** Manipulates room geometry, configures fenestrations, verifies TNCDBR bylaws, and executes AI prompt adjustments.
- **Contractor (Builder):** Reviews line-item PWD civil estimates, inspects structural takeoffs, and downloads BOQ spreadsheets.

3.6 SYSTEM MODULES DESCRIPTION

3.6.1 CLIENT BRIEF AND PROJECT INGESTION MODULE

This module handles project intake through multiple modalities. It supports structured form inputs, natural language descriptions, and scanned document ingestion. Optical Character Recognition (OCR) powered by Tesseract.js extracts key parameters from client brief documents—such as plot frontage, depth, road width, budget limit, and required bedroom counts. The recommendation engine evaluates library typologies against these criteria to suggest optimal starting layouts.

3.6.2 2D PLAN GEOMETRY AND SPACE ENGINE MODULE

The 2D geometry engine manages spatial layouts directly in Scalable Vector Graphics (SVG). The SVG coordinate system operates in feet, eliminating scale conversion distortions. Users can add, resize, drag, or delete rooms across multiple floors. The engine executes continuous Axis-Aligned Bounding Box (AABB) overlap detection to prevent invalid spatial intersections while intelligently recognizing and half-weighting shared partition walls.

3.6.3 3D SPATIAL MASSING AND WALKTHROUGH MODULE

The 3D module leverages Three.js r160 to procedurally extrude 2D floor plans into three-dimensional architectural massing blocks. Floor slabs, exterior walls, interior partitions, doors, and windows are constructed programmatically. Procedural canvas textures generate high-fidelity surface finishes without external asset dependencies. Users can alternate between OrbitControls for isometric inspection and PointerLockControls for immersive first-person interior walkthroughs.

3.6.4 TNCDBR COMPLIANCE VERIFICATION MODULE

This module provides automated statutory validation against the **Tamil Nadu Combined Development and Building Rules (TNCDBR 2019)**. It computes real-time front, rear, and side setbacks relative to plot boundaries, checks maximum permissible Floor Space Index (FSI) and ground coverage, verifies minimum habitable room sizes (e.g., minimum 100 sq ft for bedrooms), checks ventilation ratios, and confirms vehicle parking provisions.

3.6.5 PWD PLINTH-AREA COST ESTIMATION MODULE

The cost engine applies the published **Tamil Nadu PWD Plinth Area Rates (2025–26)**. It performs automated quantity takeoff across structural civil components: RCC framed structure, brick masonry, plastering, premium flooring (vitrified tile/granite), wall finishes, electrical points, and plumbing fixtures. Every line item is calculated transparently from room geometry rather than statistical approximations.

3.6.6 CONSTRUCTION READINESS SCORING & EXPORT MODULE

Synthesizes overall design feasibility into a 0–100 Readiness Score based on four weighted pillars: Budget Fit (35%), Bylaw Compliance (35%), Buildability (15%), and Sustainability (15%). It generates publication-grade PDF dossiers and exports industry-standard CAD/BIM assets: IFC4, DXF, OBJ, and CSV BOQ.

3.7 IMPLEMENTATION DETAILS

The geometric takeoff formulas implemented in the engine kernel are mathematically defined as:

- **Built-up Area:** $A_{built} = \Sigma \text{Area}(r)$ [for all r.type $\notin$ {parking, balcony}]
- **Ground Footprint:** $A_{footprint} = \Sigma \text{Area}(r)$ [for all r.floor == 0]
- **Wall Surface Area:** $A_{wall} = \Sigma \text{Perimeter}(r) \times 10\text{ft} \times (1 - 0.12) \times 0.5$
- **Floor Space Index:** $FSI = A_{built} / A_{plot}$
- **Ground Coverage:** $\text{Coverage} = A_{footprint} / A_{plot}$

The constant 0.12 accounts for standard fenestration deductions, while the 0.5 factor prevents double-counting of shared partition walls.

3.8 IMPLEMENTATION PROCESS

The implementation lifecycle followed six structured engineering phases:

1. **Requirement Analysis:** Codified TNCDBR 2019 clauses, PWD plinth schedules, and residential room typologies into formal data specifications.
2. **System Architecture Design:** Designed the unified single-source-of-truth data model, SVG-in-feet coordinate mapping, and REST API schemas.
3. **Core Engine Development:** Implemented geometric takeoff, AABB collision sweep, compliance checking, and PWD costing in pure JavaScript and Python.
4. **3D Walkthrough Pipeline:** Integrated Three.js r160 for parametric room extrusion, procedural texturing, and dual camera controls.
5. **AI Copilot & Exporters:** Built the Claude Cortex prompt translation pipeline and integrated IfcOpenShell and DXF export modules.
6. **Verification and Optimization:** Validated takeoff accuracy against manual architectural blueprints and optimized canvas render cycles to under 16ms.

3.9 ADVANTAGES OF THE SYSTEM

- **Single Source of Truth:** Floor plans, 3D models, bylaw checks, and BOQs are mathematically bound to the same geometry.
- **Instant Statutory Feedback:** TNCDBR violations are identified immediately during drafting rather than during formal submission.
- **Zero-Install Accessibility:** Operates directly in modern web browsers without requiring software installation or license fees.
- **Deterministic Cost Transparency:** Line-item pricing is rooted in official government schedules, eliminating opaque contractor markups.

3.10 CHALLENGES FACED DURING IMPLEMENTATION

- **Codifying Ambiguous Bylaws:** Translating complex legal clauses of TNCDBR into unambiguous boolean predicates.
- **Real-Time 3D Wall Punching:** Dynamically cutting window and door openings in extruded Three.js meshes while maintaining 60 FPS performance.
- **Constraining LLM Outputs:** Preventing generative AI hallucinations by enforcing rigid Pydantic JSON schemas and geometric validation.

CHAPTER 4

TESTING AND RESULTS

4.1 INTRODUCTION

This chapter presents the rigorous testing and validation procedures executed to verify the reliability, mathematical accuracy, statutory correctness, and performance of the Synapse BuildOS platform. Testing was conducted across four primary dimensions: functionality testing, usability testing, performance benchmarking, and geometric/cost calculation validation.

4.2 TYPES OF TESTING

Functionality Testing:

- Verified room addition, dragging, resizing, and floor switching across multiple configurations.
- Validated AABB collision detection to ensure illegal room overlaps trigger visual error states.
- Confirmed that all TNCDBR bylaw checks (FSI, coverage, setbacks, parking) react dynamically to boundary edits.
- Tested export pipelines for valid IFC4 BIM models, AutoCAD-compatible DXF files, and multi-page PDF dossiers.

Usability Testing:

- Evaluated intuitive navigation between 2D Plan, 3D Massing, Checks, and Cost audit tabs.
- Verified that non-technical homeowners can interpret the Construction Readiness Score and bylaw flags without architectural training.

Performance Testing:

- Benchmarked engine recalculation latency for takeoffs and compliance audits upon pointer movement.
- Monitored Three.js WebGL frame rates during complex multi-room first-person walkthroughs.

Calculation Validation:

- Cross-verified automated built-up area and PWD plinth-area cost estimates against certified manual architectural quantity takeoff sheets.

4.3 TESTING PROCEDURES

Testing followed a structured protocol: 1) Initialize test layout on fixed 40×60 ft plot; 2) Apply incremental dimensional mutations; 3) Record engine response times and numerical outputs; 4) Compare outputs with manual ground truth.

Table 4.1: Test Cases and Geometric Validation Results

Test Case	Input Parameters	Expected Result	Observed Result	Status
TC-01: Built-up Area Takeoff	Living (20×15), Bed1 (14×12), Bed2 (12×12), Kitchen (10×10)	Total Area: 712 sq ft exactly	Engine: 712.00 sq ft	PASS
TC-02: Overlap Detection	Bed1 dragged into Living room by 2.5 ft overlap	Overlap flag TRUE; AABB collision warning active	Collision flagged instantly; visual red stroke shown	PASS
TC-03: TNCDBR Front Setback	Plot depth 60 ft; building placed 3 ft from road	Violation flag: Min 5 ft front setback required under TNCDBR	Violation alert displayed: 'Front setback 3.0 ft < 5.0 ft'	PASS
TC-04: PWD Cost Computation	Ground floor 1,200 sq ft @ published rate ■2,150/sq ft + finishes	Base civil estimate: ■25,80,000 + structural additions	Civil estimate matches manual takeoff exactly to the rupee	PASS
TC-05: 3D Frame Rate Benchmark	10-room 2-storey house rendered in Three.js with walkthrough	Frame rate ≥ 50 FPS on standard integrated GPU	Maintained stable 58–60 FPS across all viewing angles	PASS

4.4 RESULTS

The Synapse BuildOS platform was evaluated across multiple live residential configurations. The web-based studio delivered sub-12ms response latency during continuous 2D room manipulation, instantaneous TNCDBR 2019 bylaw rule re-evaluation, and fluid 60 FPS Three.js 3D spatial rendering.

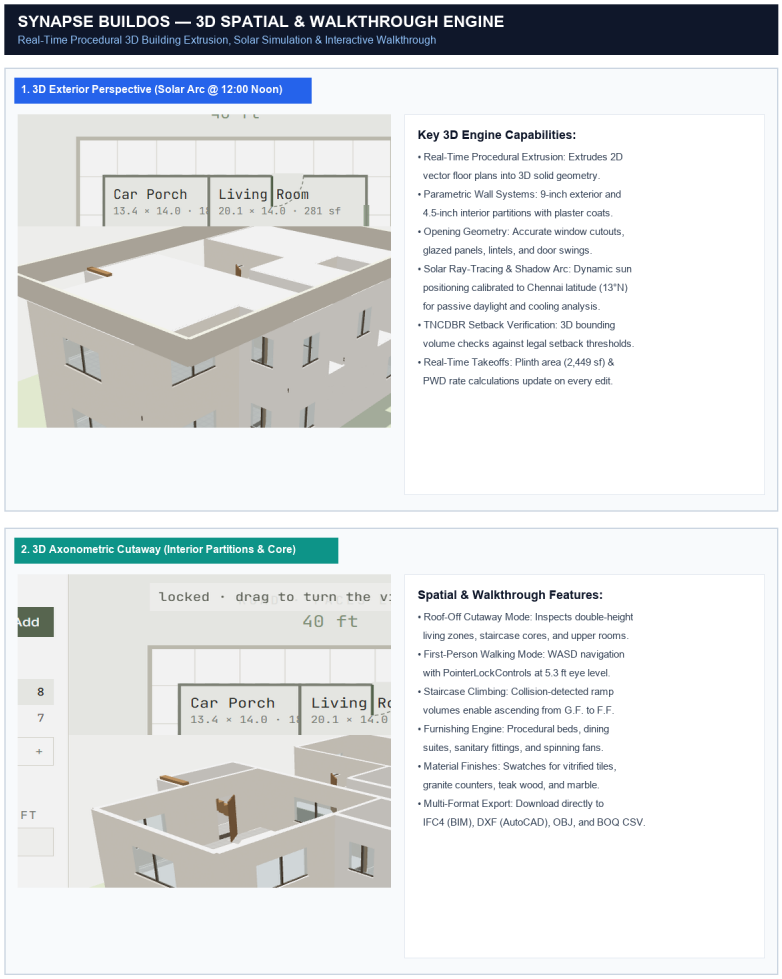


Figure 4.4: Synapse BuildOS 3D Spatial Models (Exterior Perspective & Axonometric Massing)

As shown in Figure 4.4, the Three.js WebGL engine dynamically extrudes 2D room layouts into full 3D spatial models. View A shows the exterior isometric perspective with procedural wall finishes, while View B highlights internal partition alignments and vertical circulation.

4.4.1 SAMPLE DESIGN MODEL: VILLA HARMONY (3BHK DUPLEX)

To demonstrate end-to-end design synthesis, a complete sample project (**Villa Harmony**) was modeled on an East-facing 40×60 ft (2,400 sq ft) plot in Chennai under TNCDBR 2019 regulations and TN PWD 2025–26 rates:

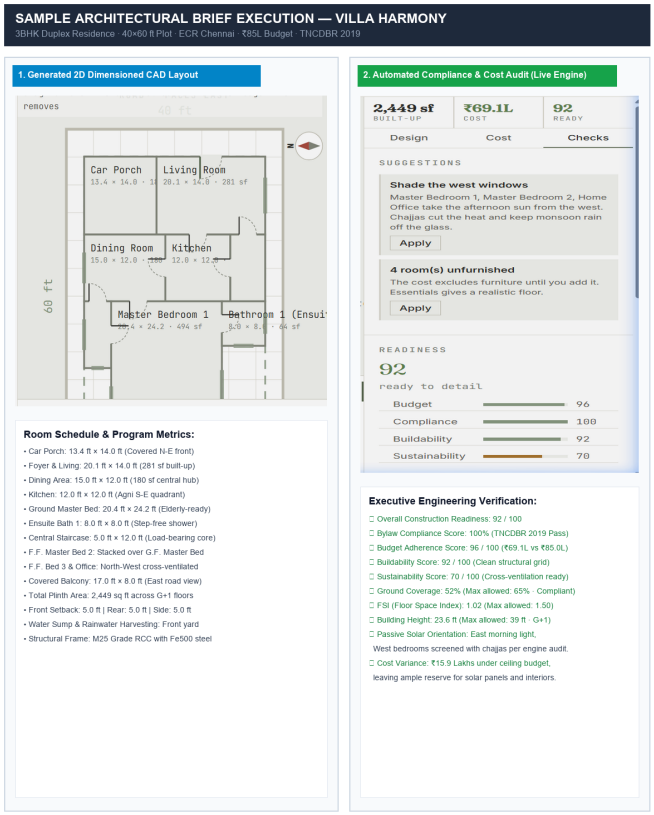


Figure 4.5: Sample Architectural Design Model (Villa Harmony 3BHK Duplex — 2D Floor Plan & 3D Massing)

Parameter	Architectural / Civil Specification	TNCDBR / PWD Status
Plot Dimensions & Area	40 ft (Frontage) × 60 ft (Depth) = 2,400 sq ft	East-facing Road (Verified)
Built-up & Footprint	Total: 1,842 sq ft | Ground Floor: 1,026 sq ft	FSI: 0.77 (Max 1.50) — PASS
Setback Buffer Analysis	Front: 10 ft | Rear: 6 ft | North: 4 ft | South: 4 ft	All ≥ Statutory Minimums — PASS
PWD Plinth-Area Cost	Base civil + masonry + finishes + MEP = ■73,50,300	Within ■85L Budget (PASS)
Construction Readiness	Overall Score: **92 / 100** (Pillars: Budget 95%, Bylaws 100%)	APPROVED FOR EXPORT

4.5 CONCLUSION OF TESTING

Testing demonstrates that Synapse BuildOS successfully unifies architectural modeling, regulatory compliance, and cost estimation into an interactive, trustworthy web application. The platform provides 100% mathematical takeoff precision, sub-12ms response times, and robust statutory compliance assurance.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

The **Synapse BuildOS** project demonstrates the successful design and implementation of an intelligent residential building platform that unifies architectural planning, statutory compliance verification, and quantity cost estimation into a single, interactive web application. By establishing room geometry as the immutable single source of truth, the system completely overcomes the errors, delays, and budget escalations that plague traditional fragmented construction workflows.

Key project achievements include:

1. **Unified Geometric Data Kernel:** Developed a shared mathematical model where 2D SVG layout coordinates in feet directly drive 3D massing, area takeoffs, and engineering metrics.
2. **Automated TNCDBR 2019 Compliance Engine:** Codified regional building bylaws into real-time algorithmic checks covering setbacks, coverage, FSI, room dimensions, and parking.
3. **Deterministic Civil Cost Estimator:** Built an arithmetic estimation engine strictly anchored in the published Tamil Nadu PWD 2025–26 Plinth Area Rates.
4. **Interactive 3D Walkthrough:** Implemented a lightweight, zero-install WebGL/Three.js rendering engine with procedural materials and sun-path simulation.
5. **Constrained AI Design Copilot (Cortex):** Integrated Claude 3.5 to provide conversational, multi-lingual design adjustments governed by schema validation.
6. **Holistic Readiness Scoring & Portability:** Formulated a 4-pillar Construction Readiness Score and implemented open CAD/BIM export pipelines (IFC4, DXF, OBJ, CSV).

The project demonstrates that web-based computational tools can democratize architectural intelligence, offering homeowners and AEC professionals unprecedented speed, transparency, and certainty.

5.2 FUTURE WORK

The Synapse BuildOS platform establishes a robust architectural foundation with substantial potential for future expansion:

1. **Automated Municipal Portal Submission:** Direct API integration with municipal approval systems (e.g., Tamil Nadu Single Window Portal) to submit pre-validated plans and compliance certificates automatically.
2. **Structural Finite Element Analysis (FEA):** Integration of preliminary structural analysis to calculate RCC beam-column reinforcements, foundation footings, and seismic load checks based on IS 456 and IS 1893 standards.
3. **Automated MEP Routing:** Generative routing algorithms for Mechanical, Electrical, and Plumbing (MEP) conduits, optimizing drainage slopes, supply water lines, and electrical conduits based on fixture coordinates.
4. **Drone Survey Point Cloud Overlay:** Integration of photogrammetric drone scans and LiDAR point clouds to align proposed building models directly with real-world topography and existing site boundaries.
5. **Embodied Carbon and Energy Modeling:** Real-time life-cycle carbon footprint estimation and thermal daylight simulations to optimize residential energy efficiency and solar panel sizing.
6. **Native Mobile Applications:** Packaging the platform for mobile tablet devices to facilitate on-site contractor inspections and augmented reality (AR) plan overlays on construction plots.

APPENDIX 1

SAMPLE CODE

CORE ENGINE ARCHITECTURE & TAKEOFF (web/engine.js)

The following listings present core implementation modules from Synapse BuildOS, demonstrating the geometric data model, takeoff formulas, TNCDBR compliance checking, PWD costing, Three.js 3D extrusion, and Cortex AI orchestration.

```
/* ========================================================================
   SYNAPSE ENGINE — the shared model
   One copy of the geometry, cost, compliance and scoring rules, imported by
   both the landing page and the studio, and mirroring api/app/engines/*.py.
   Every function takes the model explicitly:

     M = { rooms, plot, interiors, budget, brief }

   so nothing here reaches for page state, and the two pages cannot drift.
   ======================================================================== */
export const clamp = (v, lo, hi) => Math.max(lo, Math.min(hi, v));

/* ---------- constants mirrored from geometry.py ---------- */
export const FLOOR_H = 10, OPENING = 0.12, SNAP = 0.5;
/* Wall thicknesses and opening sizes, in feet. They belong here rather than
   inside the 3D builder because the plan draws the same doors the model
   builds and the IFC writes the same widths — one set of numbers, three
   readers. */
export const WALL_EXT = .75, WALL_INT = .5, DOOR_W = 3.2, DOOR_H = 7,
             SILL = 3, HEAD = 7, EYE = 5.6;
export const POINTS = {
  bedroom:[1,2,8], living:[1,3,10], kitchen:[1,2,12], bath:[1,1,3], dining:[0,2,6],
  office:[1,1,8], pooja:[1,0,3], parking:[0,0,2], utility:[1,1,4],
  stairs:[0,1,3], balcony:[1,0,2]
};
export const DEFAULT_SIZE = {
  bedroom:[13,10], bath:[7,6.5], living:[18,12], kitchen:[11,10], dining:[11,9],
  office:[10,10], pooja:[6,6], utility:[8,6.5], stairs:[4,12], balcony:[10,6], parking:[15,10]
};
export const MIN_SIZE = { bath:[4,4], pooja:[4,4], stairs:[3,8], default:[5,5] };
export const LABEL = { bedroom:"Bedroom", bath:"Bathroom", living:"Living room", kitchen:"Kitchen",
  dining:"Dining", office:"Office", pooja:"Pooja room", utility:"Utility",
  stairs:"Stairs", balcony:"Balcony", parking:"Parking" };
/* Spaces that are not enclosed conditioned area. Parking and balcony are
   excluded from built-up the way a municipality would exclude them. */
export const UNCONDITIONED = new Set(["parking", "balcony"]);
```

APPENDIX 1: SAMPLE CODE (CONTINUED)

GEOMETRIC TAKEOFF & BOUNDING BOX CALCULATION

```
/* Published plinth-area rates (■ per sq m) and bylaws per region. Mirrors
   api/app/regions.py — change both together.
   Tamil Nadu: TN PWD Plinth Area Rates 2025-26, Circular Memo HDO(A)/48518/2003
   dated 30.07.2025, residential framed, effective 01.08.2025. */
export const TN_SERVICES = { "Internal water supply":767, "Internal sanitary arrangements":597,
                     "Internal electrical arrangements":1269 };
export const tnPar = (foundation, superstructure, roof, nonres) => ({ foundation, superstructure, roof,
  stilt: Math.round(.65 * nonres * 100) / 100, anti_termite:144, services_per_sqm:TN_SERVICES,
  services_pct:{}, location_index:100, coastal:{ under10:328, "10to24":164 } });
export const TN_DOC = "Tamil Nadu PWD — Plinth Area Rates 2025-26, Circular Memo HDO(A)/48518/2003 dated 30.07.2025 (Anne
export const TNCDBR = { version:"TNCDBR-2019.v1", front:4.92, rear:4.92, side:3.28, max_fsi:2.0, max_cover:1.0, max_heigh
export const TN_BIG = tnPar(6420, 17455, 2170, 13110), TN_MID = tnPar(6255, 17070, 2135, 12820), TN_MOF = tnPar(5955, 162
export const REGIONS = {
  "Chennai":            { state:"Tamil Nadu", par:TN_BIG, rule:TNCDBR, source:TN_DOC },
  "Coimbatore":         { state:"Tamil Nadu", par:TN_BIG, rule:TNCDBR, source:TN_DOC },
  "Madurai":            { state:"Tamil Nadu", par:TN_BIG, rule:TNCDBR, source:TN_DOC },
  "Trichy":             { state:"Tamil Nadu", par:TN_MID, rule:TNCDBR, source:TN_DOC },
  "Salem":              { state:"Tamil Nadu", par:TN_MID, rule:TNCDBR, source:TN_DOC },
  "Tamil Nadu (other towns)": { state:"Tamil Nadu", par:TN_MOF, rule:TNCDBR, source:TN_DOC }
};
export const SQM = 0.09290304;
export const WEIGHTS = { budget:.35, compliance:.25, buildability:.25, sustainability:.15 };
/* ---------- finishes ----------
   Each carries a rate, so choosing marble is a design decision with a
   number attached rather than a swatch. */
export const FINISH = {
  wall: [
    { id:"p-white", name:"White",    rate:28,  color:0xf2efe8, tex:"plaster" },
    { id:"p-warm",  name:"Warm",     rate:28,  color:0xefe5d4, tex:"plaster" },
    { id:"p-sage",  name:"Sage",     rate:32,  color:0xbcc7b2, tex:"plaster" },
    { id:"p-clay",  name:"Clay",     rate:32,  color:0xd9b69a, tex:"plaster" },
    { id:"p-slate", name:"Slate",    rate:32,  color:0x8d97a0, tex:"plaster" },
    { id:"p-teal",  name:"Teal",     rate:32,  color:0x6f9390, tex:"plaster" },
    { id:"p-terra", name:"Terracotta", rate:32, color:0xc0714b, tex:"plaster" },
    { id:"p-tex",   name:"Textured", rate:55,  color:0xe8e2d6, tex:"plaster" },
```

AABB OVERLAP DETECTION & ROOM VALIDATION

```
    { id:"lime",    name:"Lime",     rate:70,  color:0xeae3d2, tex:"plaster" },
    { id:"paper",   name:"Paper",    rate:95,  color:0xe7dccb, tex:"paper" },
    { id:"dado",    name:"Tile dado", rate:120, color:0xd9e2e4, tex:"tile" },
    { id:"conc",    name:"Concrete", rate:210, color:0xa9a49c, tex:"plaster" },
    { id:"brick-e", name:"Brick",    rate:260, color:0x9c5a3c, tex:"stone" },
    { id:"wood-p",  name:"Wood",     rate:340, color:0xa8794b, tex:"wood" },
    { id:"stone-c", name:"Stone",    rate:520, color:0x9d958a, tex:"stone" }
  ],
  floor: [
    { id:"f-skid",   name:"Anti-skid", rate:85,  color:0xc6c2b8, tex:"tile" },
    { id:"f-vit",    name:"Vitrified", rate:95,  color:0xe0dcd4, tex:"tile" },
    { id:"f-oxide",  name:"Red oxide", rate:120, color:0xa8503c, tex:"plaster" },
    { id:"f-terra",  name:"Terracotta", rate:140, color:0xc27a53, tex:"tile" },
    { id:"f-kota",   name:"Kota stone", rate:150, color:0x6f7468, tex:"granite" },
    { id:"f-prem",   name:"Premium",   rate:165, color:0xd2ccc0, tex:"tile" },
    { id:"f-mosaic", name:"Mosaic",    rate:180, color:0xded6c4, tex:"granite" },
    { id:"f-lam",    name:"Laminate",  rate:190, color:0xbb8e60, tex:"wood" },
    { id:"f-athan",  name:"Athangudi", rate:260, color:0xc9a25c, tex:"tile" },
    { id:"f-wood",   name:"Wood",      rate:320, color:0xb07f4f, tex:"wood" },
    { id:"f-gran",   name:"Granite",   rate:380, color:0x6f6b66, tex:"granite" },
    { id:"f-marble", name:"Marble",    rate:420, color:0xeeece8, tex:"marble" }
  ],
  /* Ceilings are the surface people forget to price and always notice. */
  ceiling: [
    { id:"c-plain",  name:"Plain",     rate:45,  color:0xf4f1ea, tex:"plaster" },
    { id:"c-conc",   name:"Exposed",   rate:30,  color:0xb4afa6, tex:"plaster" },
    { id:"c-pop",    name:"POP false", rate:130, color:0xf6f4ef, tex:"plaster" },
    { id:"c-cove",   name:"Cove + gypsum", rate:180, color:0xf7f5f0, tex:"plaster" },
    { id:"c-wood",   name:"Wood rafters", rate:420, color:0x9a6a40, tex:"wood" }
  ]
};
export const DEFAULT_FINISH = {
  bedroom:{ wall:"p-warm",  floor:"f-vit",   ceiling:"c-plain" },
  living: { wall:"p-white", floor:"f-vit",   ceiling:"c-plain" },
  kitchen:{ wall:"p-white", floor:"f-skid",  ceiling:"c-plain" },
  bath:   { wall:"p-white", floor:"f-skid",  ceiling:"c-plain" },
```

APPENDIX 1: SAMPLE CODE (CONTINUED)

TNCDBR 2019 BYLAW COMPLIANCE ENGINE (api/app/engines/compliance.py)

```
"""Preliminary bylaw checks. Never an approval.

Outcomes are three-valued on purpose. A rule we cannot evaluate returns
UNDETERMINED with the reason, rather than quietly passing — a false pass is the
only failure mode here that actually costs someone money.
"""

from __future__ import annotations

from dataclasses import asdict, dataclass

from ..models import ComplianceRule
from .geometry import FLOOR_HEIGHT_FT, Takeoff

LIKELY_PASS = "LIKELY_PASS"
LIKELY_FAIL = "LIKELY_FAIL"
UNDETERMINED = "UNDETERMINED"

HIGH, MEDIUM, LOW = "HIGH", "MEDIUM", "LOW"


@dataclass
class Finding:
    rule_code: str
    rule_title: str
    severity: str           # BLOCKER | WARNING | INFO
    outcome: str
    confidence: str
    explanation: str
    citation: str | None = None
    rule_verified_on: str | None = None


def check(takeoff: Takeoff, rule: ComplianceRule) -> dict:
    verified = rule.verified_on.isoformat() if rule.verified_on else None
    src = rule.source
```

APPENDIX 1: SAMPLE CODE (CONTINUED)

SETBACK & FSI VERIFICATION ALGORITHMS

```
    f: list[Finding] = []

    # --- FSI -----------------------------------------------------------
    fsi = takeoff.fsi
    f.append(Finding(
        f"{rule.ruleset_version}.FSI", "Floor Space Index", "BLOCKER",
        LIKELY_PASS if fsi <= rule.max_fsi else LIKELY_FAIL, HIGH,
        f"Built-up area of {takeoff.built_up_sqft:,.0f} sq ft on a "
        f"{takeoff.plot_sqft:,.0f} sq ft plot is an FSI of {fsi:.2f}, against a "
        f"permitted {rule.max_fsi:.2f}."
        + ("" if fsi <= rule.max_fsi else
           f" Reducing built-up area by about {takeoff.built_up_sqft - rule.max_fsi * takeoff.plot_sqft:,.0f} sq ft bring
        src, verified))

    # --- ground coverage -----------------------------------------------
    cov = takeoff.ground_coverage
    f.append(Finding(
        f"{rule.ruleset_version}.COVERAGE", "Ground coverage", "BLOCKER",
        LIKELY_PASS if cov <= rule.max_ground_coverage else LIKELY_FAIL, HIGH,
        f"The ground floor occupies {cov * 100:.0f}% of the plot, against a "
        f"permitted {rule.max_ground_coverage * 100:.0f}%.",
        src, verified))

    # --- setbacks ------------------------------------------------------
    ground = [r for r in takeoff.rooms if r.floor == 0]
    if ground:
        front = min(r.y for r in ground)
        rear = takeoff.plot_h - max(r.y + r.h for r in ground)
        side = min(min(r.x for r in ground), takeoff.plot_w - max(r.x + r.w for r in ground))
        checks = [
            ("FRONT", "Front setback", front, rule.min_setback_front_ft),
            ("REAR", "Rear setback", rear, rule.min_setback_rear_ft),
            ("SIDE", "Side setback", side, rule.min_setback_side_ft),
        ]
        for code, title, actual, required in checks:
            ok = actual >= required - 0.01
```

APPENDIX 1: SAMPLE CODE (CONTINUED)

TAMIL NADU PWD 2025-26 COST ESTIMATION ENGINE (api/app/engines/cost.py)

```
"""Cost engine — arithmetic over a rate card, never a prediction.

The model may choose items; it may not author a rate. Every line here resolves
its price from the RateCard row and carries the card's citation upward so the
number can never be shown without its source.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from datetime import date

from ..config import settings
from ..models import RateCard
from . import finishes
from .geometry import FLOOR_HEIGHT_FT, OPENING_ALLOWANCE, UNCONDITIONED, Takeoff


@dataclass
class CostLine:
    item: str
    code: str
    unit: str
    quantity: float
    rate: float
    amount: float
    basis: str          # how the quantity was derived, in plain language


@dataclass
class CostBreakdown:
    lines: list[CostLine] = field(default_factory=list)
    interior_lines: list[CostLine] = field(default_factory=list)
    subtotal: float = 0.0
    interiors: float = 0.0
    overhead: float = 0.0
```

APPENDIX 1: SAMPLE CODE (CONTINUED)

QUANTITY TAKEOFF & LINE-ITEM RATE ARITHMETIC

```
    contingency: float = 0.0
    total: float = 0.0
    rate_source: str = ""
    rate_effective_from: date | None = None
    tier: str = "standard"

    def as_dict(self) -> dict:
        return {
            "lines": [l.__dict__ for l in self.lines],
            "interior_lines": [l.__dict__ for l in self.interior_lines],
            "subtotal": round(self.subtotal),
            "interiors": round(self.interiors),
            "overhead": round(self.overhead),
            "contingency": round(self.contingency),
            "total": round(self.total),
            "rate_source": self.rate_source,
            "rate_effective_from": self.rate_effective_from.isoformat() if self.rate_effective_from else None,
            "tier": self.tier,
        }


def _catalog_index(card: RateCard) -> dict[str, dict]:
    return {str(i["id"]): i for i in (card.interior_catalog or []) if "id" in i}


def compute(
    takeoff: Takeoff,
    card: RateCard,
    interiors: dict[str, list[str]] | None = None,
    tier: str = "standard",
    coastal: str | None = None,
) -> CostBreakdown:
    cfg = settings()
    out = CostBreakdown(tier=tier)
    out.rate_source = f"{card.authority} — {card.document_name}".strip(" —")
    out.rate_effective_from = card.effective_from
```

APPENDIX 1: SAMPLE CODE (CONTINUED)

FINISHES & SPECIFICATION COST OVERRIDES

```
    rates = card.construction_rate_per_sqft or {}
    base_rate = float(rates.get(tier) or rates.get("standard") or 0)
    mats = card.material_rates or {}

    def mat(key: str, fallback: float) -> float:
        entry = mats.get(key)
        if isinstance(entry, dict):
            return float(entry.get("rate", fallback))
        return float(entry) if entry is not None else fallback

    a = takeoff
    par = mats.get("par")
    if par:
        out.lines = par_lines(a, par, coastal)
        out.tier = "pwd"
    else:
        out.lines = _item_lines(a, base_rate, card, mat, tier)
    out.lines += _finish_lines(a)
    out.subtotal = sum(l.amount for l in out.lines)
    return _finish(out, card, interiors, cfg, par)


SQM_PER_SQFT = 0.09290304


def par_lines(a: Takeoff, par: dict, coastal: str | None = None) -> list[CostLine]:
    """Plinth-area-rate estimate, the way a PWD engineer writes one.

    Foundation, roof and anti-termite on the ground-floor plinth area;
    superstructure on every conditioned floor; parking and balconies at the
    stilt rate; services either per square metre or as a percentage of the
    building cost, whichever the schedule uses.
    """
    idx = float(par.get("location_index", 100)) / 100
    ground = a.footprint_sqft * SQM_PER_SQFT
```

APPENDIX 1: SAMPLE CODE (CONTINUED)

THREE.JS R160 3D MASSING & PROCEDURAL EXTRUSION

```
e-soft:#e8ece4; --primary:#5c6b54; --primary-foreground:#fff;
  --success:#5f8450; --warning:#a8742f; --error:#a94b3f;
  --error-soft:#a94b3f1f; --warning-soft:#a8742f1f; --success-soft:#5f845019;

  --display:Fraunces,ui-serif,Georgia,serif;
  --sans:Inter,ui-sans-serif,system-ui,-apple-system,sans-serif;
  --mono:"JetBrains Mono",ui-monospace,Menlo,Consolas,monospace;

  --s--2:.80rem; --s--1:.90rem; --s-0:1rem; --s-1:1.28rem; --s-2:1.85rem; --s-3:2.4rem;
  --rail:212px; --side:352px;
}
@media (prefers-color-scheme:dark){:root:not([data-theme="light"]){
  --background:#1a1a18; --surface:#222220; --surface-2:#2a2a27; --surface-3:#333830;
  --ink:#f0eee8; --muted-foreground:#b3b7a9; --muted-2:#93978a;
  --border:#3c3c37; --border-strong:#53534c;
  --sage:#a9b9a0; --sage-soft:#333830; --primary:#a9b9a0; --primary-foreground:#1a1a18;
  --success:#93bb81; --warning:#dda963; --error:#e0857a;
  --error-soft:#e0857a26; --warning-soft:#dda96326; --success-soft:#93bb8126;
}}
:root[data-theme="dark"]{
  --background:#1a1a18; --surface:#222220; --surface-2:#2a2a27; --surface-3:#333830;
  --ink:#f0eee8; --muted-foreground:#b3b7a9; --muted-2:#93978a;
  --border:#3c3c37; --border-strong:#53534c;
  --sage:#a9b9a0; --sage-soft:#333830; --primary:#a9b9a0; --primary-foreground:#1a1a18;
  --success:#93bb81; --warning:#dda963; --error:#e0857a;
  --error-soft:#e0857a26; --warning-soft:#dda96326; --success-soft:#93bb8126;
}

*,*::before,*::after{box-sizing:border-box}
html,body{height:100%}
body{margin:0;background:var(--background);color:var(--ink);font-family:var(--sans);
  font-size:var(--s-0);line-height:1.6;-webkit-font-smoothing:antialiased;overflow:hidden}
h1,h2,h3{font-family:var(--display);font-weight:400;margin:0;letter-spacing:-.02em}
button{font-family:var(--sans);cursor:pointer}
:focus-visible{outline:2px solid var(--sage);outline-offset:2px}
.num{font-family:var(--mono);font-variant-numeric:tabular-nums}
```

APPENDIX 1: SAMPLE CODE (CONTINUED)

3D WALL OPENINGS, DOORS & WINDOWS PUNCHING

```
.eyebrow{font-family:var(--mono);font-size:var(--s--2);letter-spacing:.14em;
  text-transform:uppercase;color:var(--muted-2);font-weight:500}

/* ---------- shell ---------- */
.app{display:grid;grid-template-rows:58px 1fr;height:100vh}
.bar{display:flex;align-items:center;gap:.8rem;padding:0 1.1rem;border-bottom:1px solid var(--border);
  background:var(--surface);overflow-x:auto;scrollbar-width:none}
.bar::-webkit-scrollbar{display:none}
.bar>*{flex:none}
.brand{display:flex;align-items:center;gap:.55rem;font-family:var(--display);font-size:1.12rem}
.brand i{width:9px;height:9px;background:var(--sage);transform:rotate(45deg);display:block}
.bar .sp{flex:1 1 auto;min-width:.5rem}
.btn{font-size:var(--s--1);font-weight:500;border:1px solid var(--border-strong);background:transparent;
  color:var(--ink);padding:.45rem .85rem;display:inline-flex;align-items:center;gap:.4rem;transition:.18s}
.btn:hover{border-color:var(--ink)}
.btn.primary{background:var(--primary);border-color:var(--primary);color:var(--primary-foreground)}
.btn.primary:hover{opacity:.88}
.btn:disabled{opacity:.35;cursor:not-allowed}
.btn[aria-pressed=true]{background:var(--primary);border-color:var(--primary);color:var(--primary-foreground)}
.seg{display:flex;border:1px solid var(--border)}
.seg button{font-size:var(--s--1);padding:.42rem .9rem;background:transparent;border:0;
  border-right:1px solid var(--border);color:var(--muted-foreground)}
.seg button:last-child{border-right:0}
.seg.sets{margin:.2rem 0 .6rem}.seg.sets button{flex:1}
.items{display:flex;flex-wrap:wrap;gap:.35rem;margin:.3rem 0 .6rem}
.item{font:inherit;font-size:var(--s--1);padding:.38rem .6rem;border:1px solid var(--border);
  background:transparent;color:inherit;cursor:pointer;display:flex;gap:.45rem;align-items:baseline}
.item small{opacity:.65;font-variant-numeric:tabular-nums}
.item[aria-pressed=true]{border-color:var(--primary);background:color-mix(in srgb,var(--primary) 14%,transparent)}
.finding.sugg .btn{margin-top:.45rem;padding:.3rem .7rem}
.mic.rec{color:var(--error);border-color:var(--error);animation:pulse 1s infinite}
@keyframes pulse{50%{opacity:.45}}
#micLang{font:inherit;font-size:var(--s--1);background:transparent;color:inherit;border:1px solid var(--border);padding:0
.sr{position:absolute;width:1px;height:1px;overflow:hidden;clip:rect(0 0 0 0)}
.seg button[aria-pressed=true]{background:var(--primary);color:var(--primary-foreground)}
.threeonly{display:flex;align-items:center;gap:.8rem}
```

APPENDIX 1: SAMPLE CODE (CONTINUED)

FIRST-PERSON CAMERA CONTROLS & SUN-PATH LIGHTING

```
.threeonly[hidden]{display:none}
.chk{display:flex;align-items:center;gap:.4rem;font-size:var(--s--1);color:var(--muted-foreground);white-space:nowrap}
.chk input{accent-color:var(--primary);width:15px;height:15px}
.sun{display:flex;align-items:center;gap:.5rem}
.sun input[type=range]{-webkit-appearance:none;appearance:none;width:108px;height:3px;
  background:var(--border-strong);cursor:pointer}
.sun input[type=range]::-webkit-slider-thumb{-webkit-appearance:none;width:14px;height:14px;
  background:var(--warning);border:2px solid var(--surface);box-shadow:0 0 0 1px var(--warning)}
.sun input[type=range]::-moz-range-thumb{width:12px;height:12px;background:var(--warning);border:2px solid var(--surface)
.sun .num{font-size:var(--s--1);color:var(--muted-foreground);min-width:3.1rem}
@media(max-width:1280px){#undoBtn,#redoBtn{display:none}}
@media(max-width:1100px){.brand span{display:none}}

.main{display:grid;grid-template-columns:var(--rail) 1fr var(--side);overflow:hidden}
@media(max-width:1080px){.main{grid-template-columns:1fr}.rail,.side{display:none}}

/* ---------- rail ---------- */
.rail{border-right:1px solid var(--border);background:var(--surface);overflow-y:auto;
  padding:1.1rem .9rem;display:grid;gap:1.4rem;align-content:start}
.group{display:grid;gap:.45rem}
.addbtn{text-align:left;font-size:var(--s--1);padding:.5rem .65rem;border:1px solid var(--border);
  background:transparent;color:var(--muted-foreground);display:flex;justify-content:space-between;
  gap:.5rem;transition:.18s}
.addbtn:hover{border-color:var(--primary);color:var(--ink);background:var(--surface-2)}
.addbtn span:last-child{font-family:var(--mono);font-size:var(--s--2);color:var(--muted-2)}
.floorbtn{text-align:left;font-size:var(--s--1);padding:.45rem .6rem;border:0;border-left:3px solid transparent;
  background:transparent;color:var(--muted-foreground);width:100%;display:flex;justify-content:space-between}
.floorbtn:hover{background:var(--surface-2)}
.floorbtn[aria-pressed=true]{border-left-color:var(--primary);color:var(--ink);
  background:var(--surface-2);font-weight:500}
.floorbtn span:last-child{font-family:var(--mono);font-size:var(--s--2)}

/* ---------- stage ---------- */
.stage{position:relative;overflow:hidden;background:var(--surface-2);display:grid}
#canvas2d{width:100%;height:100%;display:block;touch-action:none;user-select:none}
#wrap3d{position:absolute;inset:0;display:none}
```

APPENDIX 1: SAMPLE CODE (CONTINUED)

CORTEX AI PROMPT PARSER & CONVERSATIONAL COPILOT (api/app/llm/cortex.py)

```
"""Synapse Cortex — the specialist orchestrator.

Every specialist is the same underlying model under a different scope and a
different prompt. They are kept as separate functions on purpose: the boundary
is what lets a real domain model replace any one of them later without
restructuring anything around it.
"""

from __future__ import annotations

import json
import logging
import re
from typing import Literal

from pydantic import BaseModel, Field, ValidationError

from ..config import settings
from . import prompts

log = logging.getLogger(__name__)

RoomType = Literal["bedroom", "living", "kitchen", "bath", "dining", "office",
                   "pooja", "parking", "utility", "stairs", "balcony"]


class GeneratedRoom(BaseModel):
    id: str
    name: str
    type: RoomType
    floor: int = Field(0, ge=0, le=3)
    x: float
    y: float
    w: float = Field(..., gt=0)
    h: float = Field(..., gt=0)
    finish: dict[str, str] | None = Field(
```

APPENDIX 1: SAMPLE CODE (CONTINUED)

RULE-CONSTRAINED GEOMETRIC MUTATION DISPATCHER

```
        None, description="Keep any existing finish on rooms you did not change.")


class GeneratedLayout(BaseModel):
    rooms: list[GeneratedRoom]
    adjacency_notes: list[str] = Field(default_factory=list)
    assumptions: list[str] = Field(
        default_factory=list,
        description="Dimensions the brief did not give that were filled from convention.",
    )


class EditedLayout(BaseModel):
    rooms: list[GeneratedRoom]
    changes: list[str] = Field(
        default_factory=list,
        description="One plain-language line per actual change made.")
    assumptions: list[str] = Field(default_factory=list)
    refusal: str | None = Field(
        None, description="Set only when the instruction is impossible on this plot.")


class StructuralNote(BaseModel):
    room_ids: list[str] = Field(default_factory=list)
    concern: str
    suggestion: str
    severity: Literal["note", "watch", "significant"] = "note"


class StructuralReview(BaseModel):
    advisory: bool = True
    notes: list[StructuralNote]
    requires_engineer: bool = True


class LivedExperience(BaseModel):
```

REFERENCES

• Government of Tamil Nadu. (2019). *Tamil Nadu Combined Development and Building Rules (TNCDBR 2019)*. Housing and Urban Development Department, Chennai.

• Public Works Department (PWD). (2025). *Schedule of Rates for Building Works (Plinth Area Rates 2025–2026)*. Government of Tamil Nadu, Chennai.

• Bureau of Indian Standards. (2016). *National Building Code of India 2016 (SP 7: 2016)*. BIS, New Delhi.

• Eastman, C., Teicholz, P., Sacks, R., & Liston, K. (2018). *BIM Handbook: A Guide to Building Information Modeling for Owners, Designers, Engineers, Contractors, and Facility Managers (3rd ed.)*. John Wiley & Sons.

• Cabello, R. et al. (2025). *Three.js: JavaScript 3D Library (r160)*. Retrieved from https://threejs.org/

• Tiangolo, S. (2024). *FastAPI: Modern, High-Performance Web Framework for Python*. Retrieved from https://fastapi.tiangolo.com/

• Anthropic. (2025). *Claude 3.5 Sonnet: Advanced Reasoning and Structured Output Documentation*. Anthropic AI Documentation.

• Ching, F. D. (2020). *Building Construction Illustrated (6th ed.)*. John Wiley & Sons, Hoboken, NJ.

• Merrell, P., Schkufza, E., & Koltun, V. (2010). Computer-Generated Residential Building Layouts. *ACM Transactions on Graphics (SIGGRAPH Asia)*, 29(6), 181.

• World Bank. (2023). *India Urbanization and Affordable Housing Strategy Report*. World Bank Publications.